

Projet Quiz — Audit des exercices de cours

Ce qui manque et ce qu'il reste à faire, comparé aux 4 supports de cours (NodeJS API REST, Gestion de fichiers, Communication instantanée, CI/CD) — mis à jour le 27 juillet 2026

Résumé exécutif

Le volet **CI/CD a été traité** depuis le dernier audit : docker-compose.yml a maintenant un service base de données et un service app avec live-reload, et des pipelines CI (build + tests) et CD (publication de l'image) ont été ajoutés dans .github/workflows/. Le thème « Gestion des fichiers » reste intégralement conforme, bonus compris. Il reste deux manques structurels sur les autres volets : aucune validation d'entrée (pas de class-validator) et **aucune authentification** nulle part dans l'app (ni HTTP ni WebSocket). Par ailleurs, 3 actions manuelles côté CI/CD ne peuvent pas être automatisées par un simple commit de fichiers — voir la fin de la section 4.

1. NodeJS 2 — Développer une API REST

Exercice / point	Statut	Manque / action à faire
Ex.1.1 — Express + endpoint création utilisateur	FAIT	—
Ex.1.2 — ORM + Datasource + entité User	PARTIEL	Prisma utilisé à la place de TypeORM (demandé dans le cours). Fonctionnellement équivalent, mais à mentionner en soutenance / documenter le choix.
Ex.1.3 / 1.4 — Tester l'endpoint avec Bruno + créer la requête Bruno	MANQUANT	Aucun fichier .bru versionné dans le repo (seul un PDF de plan de test existe). → Créer et committer une vraie collection Bruno (dossier bruno/).
Ex.2 — 2e entité + controllers + tests Bruno	PARTIEL	Entités Question/Media/Answer et routes présentes. Toujours pas de collection Bruno versionnée pour ces routes.
Ex.3.1 — class-validator pour valider les entrées	MANQUANT	Pas de dépendance class-validator, validation faite à la main (if manquant). → Installer class-validator (ou équivalent : zod/joi) et valider chaque body reçu.
Ex.3.2 — Tests Bruno prouvant la validation	MANQUANT	Dépend du point précédent.
Bonus 1 — Validation en middleware Express	MANQUANT	→ Créer un middleware générique de validation réutilisable sur toutes les routes POST/PATCH.
Bonus 2 — Autres entités/endpoints nécessaires	FAIT	—
Hors exercice mais annoncé au planning — Authentification	MANQUANT	Aucun champ mot de passe, aucun JWT, aucune route de login/register, aucun middleware d'auth. → Bloquant pour sécuriser users, submit, WebSocket.

2. NodeJS 4 — Gérer les fichiers

Thème entièrement conforme, bonus inclus : upload + redimensionnement (Sharp), entité Media, stockage S3 via Minio, et transformation à la volée avec cache Redis (/api/media/:id/transform). Aucune action requise.

3. NodeJS 5 — Communication instantanée

Exercice / point	Statut	Manque / action à faire
Consigne principale — Réponses au quiz en temps réel	FAIT	Socket.IO opérationnel (événement quiz:answer).
Point technique — Classe dédiée à la gestion des WS	MANQUANT	Logique inline dans server.ts. → Extraire dans une classe/module dédié (ex. src/ws/QuizGateway.ts) pour la maintenabilité.
Point technique — Authentification sur le canal WS	MANQUANT	N'importe qui peut se connecter et soumettre une réponse. → Vérifier un token/session à la connexion Socket.IO (middleware io.use()).
Bonus — Validation entrée/sortie (HTTP et WS)	MANQUANT	→ Réutiliser la même librairie de validation que pour les endpoints REST.
Bonus — Gestion de salons (rooms)	MANQUANT	Toutes les connexions partagent le même flux. → Introduire des rooms par partie/quiz.
Bonus — Solution de test alternative à Bruno pour les WS	MANQUANT	→ Mettre en place un client de test (ex. socket.io-client dans un script/test).

4. CI/CD

Exercice / point	Statut	Manque / action à faire
Ex.1 — Dockerfile copie code + npm install + run	FAIT	Dépassé : le Dockerfile va déjà jusqu'au build final.
Ex.3 — Build stages (image optimisée, comparaison poids)	FAIT	Dockerfile multi-stage (builder + image finale allégée) déjà en place.
Ex.4 — Live-reloading (volume src monté)	FAIT	Service app ajouté dans docker-compose.yml (build sur le stage builder, commande npm run dev) + volumes ./src, ./public, ./prisma montés.
Planning — Passage vers une vraie orchestration (BDD incluse)	FAIT	Service db (MySQL 8, conforme au provider prisma) ajouté avec volume de persistance et healthcheck dans docker-compose.yml.
Planning — Mise en place d'une CI (build + qualité de code)	FAIT	.github/workflows/ci.yml : install, prisma generate, build TypeScript, tests, build de l'image Docker, à chaque push/PR.
Planning — Exécution des tests en CI	FAIT	Tests Jest/Supertest ajoutés (tests/) et branchés dans la CI. Prisma est mocké : pas besoin de base de données pour les faire tourner.
Planning — Protection des branches	MANQUANT	Réglage du dépôt GitHub, pas un fichier versionnable → action manuelle (voir « Actions manuelles restantes » ci-dessous).
Planning — Mise en place d'une CD	FAIT	.github/workflows/cd.yml : build + push de l'image vers GHCR, déclenché uniquement si la CI a réussi sur main.
Planning — Documentation du processus + recette	FAIT	README.md réécrit : setup local, Docker/Compose, tests, CI/CD, procédure de recette.

Actions manuelles restantes (non automatisables par fichier)

Exercice / point	Statut	Manque / action à faire
npm install + commit du package-lock.json	MANQUANT	Jest/ts-jest/supertest ont été ajoutés à package.json mais pas installés depuis cet environnement (pas d'accès au registre npm). npm ci échouera en CI tant que le lockfile n'est pas régénéré chez toi et commité.
Vérifier docker compose up -d --build en local	MANQUANT	Le fichier a été validé syntaxiquement (YAML) mais jamais réellement lancé (pas de Docker disponible dans cet environnement).
Activer la protection de la branche main sur GitHub	MANQUANT	Settings → Branches → règle exigeant que le check CI passe avant de merger (étapes détaillées dans le README).

Plan d'action priorisé

Priorité	Action	Pourquoi
Haute	Lancer npm install en local et commit le package-lock.json mis à jour	Sans ça, npm ci échoue en CI (jest/ts-jest/supertest ajoutés mais jamais installés)
Haute	Vérifier docker compose up -d --build en local	Le compose n'a jamais tourné réellement (pas de Docker dans l'environnement d'audit)
Haute	Activer la protection de la branche main sur GitHub	Réglage GitHub, à faire manuellement (étapes dans le README)
Haute	Mettre en place l'authentification (login + JWT/session) et l'appliquer sur les routes REST et le WebSocket	Aucune protection actuelle sur users, submit, media, WS
Haute	Ajouter la validation d'entrée (class-validator ou équivalent) en middleware réutilisable	Toutes les routes acceptent des données non validées
Moyenne	Versionner une vraie collection Bruno (.bru) au lieu du PDF de plan de test	Demandé explicitement par les exercices 1 et 2 du cours API REST
Moyenne	Extraire la logique WebSocket dans une classe dédiée + ajouter la gestion de rooms	Maintenabilité, bonus du cours WS

Document généré à partir d'une comparaison entre le code du repository Quiz (github.com/phasna/Quiz) et les 4 supports de cours fournis.